Text Classification Battle

Decision Tree vs KNN vs SVM vs Naïve Bayes on BBC News

Information Retrieval (21CSE377T) – CLA-2, Task 2

Name: K R Vaishak
Reg No: RA2411026050018
Class: CSE AIML A
Year: 3rd Year
Faculty: Dr. S. Prabu

GitHub Repository: https://github.com/Vaishakkr/IR-Text-Classification-Battle

1. Problem Statement

A news website receives hundreds of articles every day, and each one has to be filed under the right section before readers can find it. Doing that by hand does not scale, so the task is treated as a supervised text-classification problem: given the text of an article, predict its section. Four classical classifiers – Decision Tree, K-Nearest Neighbours, Support Vector Machine and Multinomial Naïve Bayes – are trained on exactly the same data and features, so that any difference in the results can be attributed to the algorithm itself. This links to SDG 9 (Industry, Innovation and Infrastructure): automatic routing of documents is a basic building block of digital information infrastructure such as news portals, search engines and enterprise document systems.

2. Dataset and Class Definition

The BBC News corpus released by Greene and Cunningham (ICML 2006) was used. It contains 2,225 full-text articles published on the BBC News website in 2004–2005, each manually assigned to one of five topical areas. It is a real, publicly available benchmark and not a synthetic dataset.

Class	What it covers
business	companies, markets, economy, banking
entertainment	film, music, television, awards, celebrities
politics	UK parliament, parties, elections, government policy
sport	football, rugby, tennis, athletics, cricket
tech	computing, internet, mobile, games, gadgets

While checking the data, 99 articles were found to appear twice with identical text. Keeping them would allow the same article to sit in both the training set and the test set, which inflates scores, so the duplicates were removed. The final dataset has 2,126 documents:

Class	Documents	Percent
sport	504	23.7100
business	503	23.6600
politics	403	18.9600

entertainment	369	17.3600
tech	347	16.3200

The ratio between the largest class (sport, 504) and the smallest (tech, 347) is 1.45. This is mild imbalance, but it is enough that plain accuracy could hide weak performance on the smaller classes. For that reason macro-averaged precision, recall and F1 (every class weighted equally) are the main metrics, and the train/test split is stratified so each class keeps the same share in both parts.

3. Methodology

3.1 Preprocessing

Every article goes through the same four steps: (i) convert to lowercase; (ii) keep only alphabetic tokens using the regular expression [a-z]+, which drops numbers and punctuation; (iii) remove English stop words (scikit-learn's built-in list of 318 words) and tokens of two letters or fewer; (iv) reduce each word to its stem with the Porter stemmer, so that 'market', 'markets' and 'marketing' share one feature. The average article shrinks from 390.5 words to 194.6 tokens. One example:

```
CLASS : tech

BEFORE (first 60 words):
tv future in the hands of viewers with home theatre systems plasma high-definition tvs and
digital video recorders moving into the living room the way people watch tv will be radically
different in five years time. that is according to an expert panel which gathered at the
annual consumer electronics show in las vegas to discuss how these new technologies

AFTER (first 40 tokens):
futur hand viewer home theatr system plasma high definit tv digit video record move live room
way peopl watch radic differ year time accord expert panel gather annual consum electron la
vega discuss new technolog impact favourit pastim lead trend
```

3.2 Feature representation – TF-IDF

The cleaned text is converted into TF-IDF vectors with these settings: min_df = 2 (a term must occur in at least two training documents), max_df = 0.95 (terms found in almost every document are dropped) and sublinear_tf = True (term frequency is replaced by 1 + log tf, so a word repeated twenty times does not count twenty times as much). Each document vector is L2-normalised. The vectoriser is fitted on the training documents only and then applied to the test documents, so no vocabulary or IDF information leaks from the test set. The result is a 1,700 × 9,881 training matrix that is 98.74% zeros.

3.3 Experimental protocol

- Stratified 80/20 split: 1,700 training and 426 test documents, random_state = 42.
- The identical TF-IDF matrix is passed to all four models.
- Models: DecisionTreeClassifier (default, fully grown); KNeighborsClassifier (k = 5, cosine distance); LinearSVC (C = 1.0); MultinomialNB (alpha = 1.0, Laplace smoothing). Default-style settings were used on purpose so the comparison reflects the algorithms rather than tuning effort.
- Metrics on the test set: accuracy, macro precision, macro recall, macro F1, confusion matrix, per-class F1, training time and prediction time.

- A 5-fold stratified cross-validation on the training set (TF-IDF refitted inside every fold) was added to check that the single test split is not giving a lucky or unlucky picture.
- Implementation: Python 3, scikit-learn 1.9, NLTK (Porter stemmer), pandas, matplotlib. The complete run (python main.py) takes about 15 seconds on a laptop.

4. Results

4.1 Comparative table

Model	Accuracy	Precision	Recall	F1	Train acc.	CV F1 mean	CV F1 std	Correct / 426
Decision Tree	0.8310	0.8285	0.8186	0.8222	1.0000	0.8045	0.0237	354
KNN (k=5)	0.9742	0.9737	0.9719	0.9727	0.9741	0.9544	0.0084	415
SVM (Linear)	**0.9789**	**0.9795**	**0.9782**	**0.9787**	**1.0000**	**0.9774**	**0.0043**	**417**
Naive Bayes	0.9507	0.9576	0.9433	0.9487	0.9824	0.9575	0.0038	405

Model	Train time (s)	Predict time (s)	Features used
Decision Tree	0.2913	0.0009	123 of 9,881 (tree splits)
KNN (k=5)	0.0026	0.0249	all 9,881
SVM (Linear)	0.0468	0.0008	all 9,881
Naive Bayes	0.0046	0.0007	all 9,881

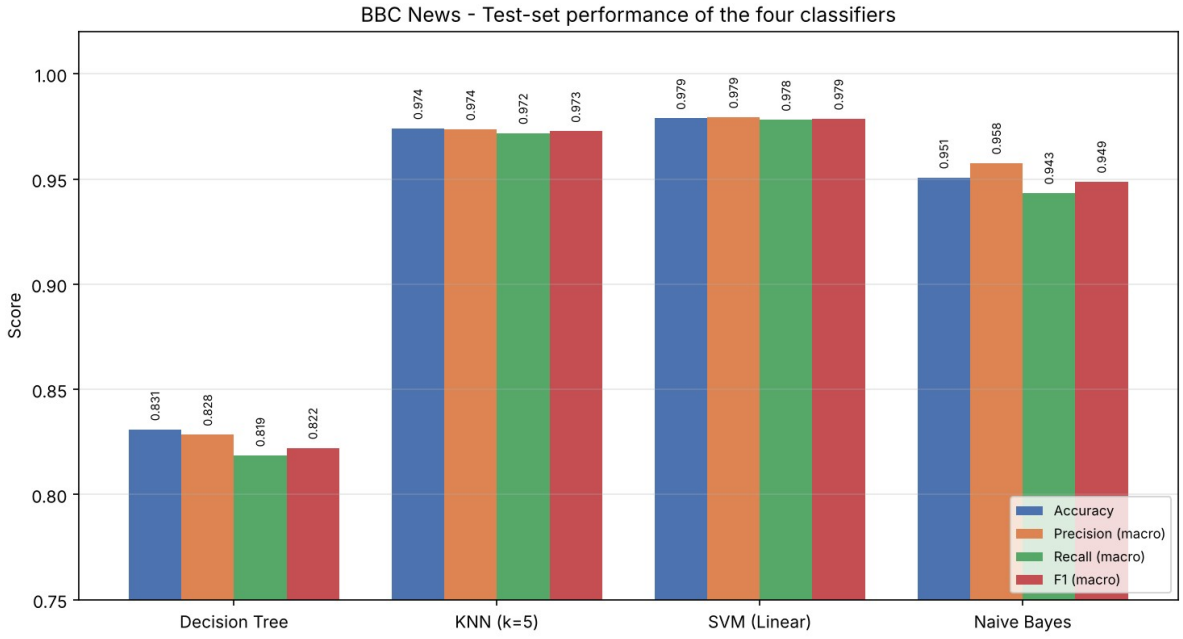


Figure 1: Accuracy, precision, recall and F1 of the four classifiers on the test set

4.2 Per-class F1

Class	Decision Tree	KNN (k=5)	SVM (Linear)	Naive Bayes

business	0.8224	0.9655	0.9608	0.9346
entertainment	0.7552	0.9726	0.9733	0.9362
politics	0.8387	0.9816	0.9811	0.9390
sport	0.9275	0.9951	1.0000	0.9951
tech	0.7669	0.9489	0.9781	0.9385

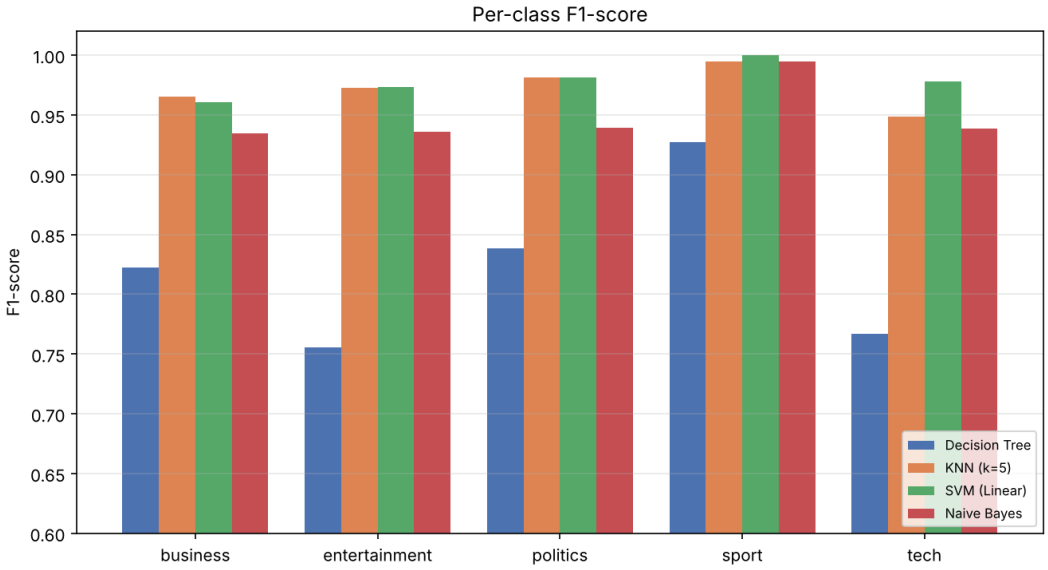


Figure 2: Per-class F1-score

4.3 Confusion-matrix analysis

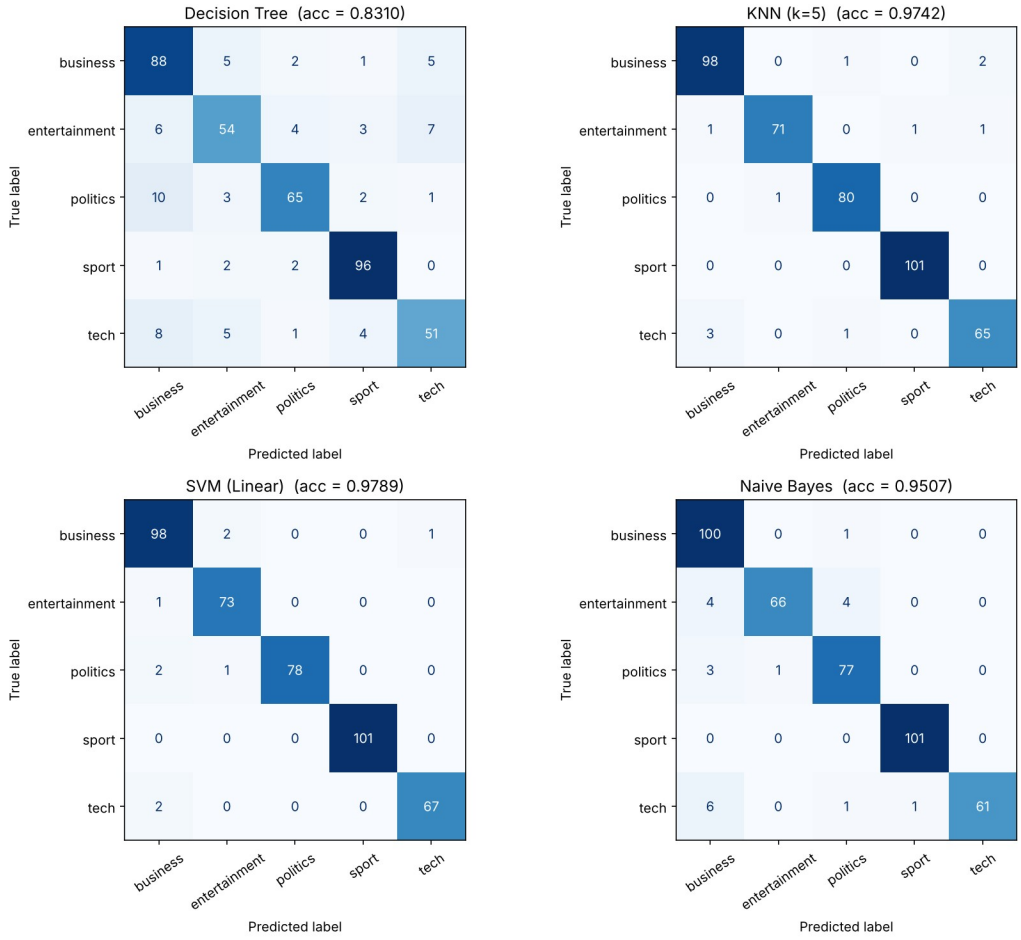


Figure 3: Confusion matrices (rows = true class, columns = predicted class)

- **Sport is almost solved.** sport is the easiest class for every model (SVM and KNN get all 101 right). Sports articles use a vocabulary – match, coach, cup, rugbi, olymp – that almost never shows up in other sections.
- **Business is the confusing hub.** Most mistakes involve business. In the SVM matrix, 6 of the 9 errors either come from business or are pushed into it. Business news overlaps with every other section: a games company being sold, a fuel-duty decision by the chancellor, or the state of the German music industry are all partly business stories.
- **Naïve Bayes over-predicts business.** Naïve Bayes predicts business 113 times although only 101 test articles are business; 13 articles from entertainment, politics and tech are wrongly pulled in. This is why its business recall is 0.99 while its business precision drops to 0.885.
- **Decision Tree errors are spread everywhere.** The Decision Tree's errors are scattered across almost every cell, especially entertainment → tech (7) and politics → business (10). It is not confusing one specific pair; it is generally unreliable.

The nine articles the best model (SVM) gets wrong were read individually (results/misclassified_best_model.csv). Examples: 'Games maker fights for survival' (tech, predicted business) is about a company going up for sale; 'Petrol duties frozen, Brown says' (politics, predicted

business) is about tax policy; 'German music in a zombie state' (entertainment, predicted business) is about record-industry revenues. In these cases a human editor could reasonably file the article either way, so the remaining errors come mostly from genuine topic overlap rather than from a defect in the model.

5. Why the Classifiers Behave Differently

Linear SVM (best, F1 0.9787). Text in TF-IDF form has thousands of dimensions but each class is described by a handful of strong words, which makes the classes close to linearly separable. A linear SVM looks for the boundary with the widest margin between classes and gives every one of the 9,881 terms a weight, so many weak clues can add up. Its top weights are clean, discriminative words (business: bank, firm, compani, econom; politics: parti, minist, labour, blair). It reaches 100% training accuracy, yet the margin and the C = 1 regularisation keep it from memorising noise, and it holds 97.9% on unseen articles with the lowest CV spread (±0.0043).

KNN (F1 0.9727). KNN labels an article by a vote of its five most similar training articles. Since TF-IDF plus cosine similarity is precisely the ranking idea used in Task 1, articles on the same topic sit close together and KNN does very well. It has no real training phase (2.6 ms), but at prediction time it must compare every test article with all 1,700 training articles, which makes it the slowest model to predict (about 30× slower than SVM here, and the gap grows with collection size). Its CV score (0.954) is noticeably lower than its test score, showing it is more sensitive to which documents happen to be its neighbours.

Naïve Bayes (F1 0.9487). Multinomial NB is a generative model: it estimates how likely each word is inside each class and assumes words are independent. It trains in about 5 ms and is very stable in CV (±0.0038). Its weakness is visible in its highest-probability words: 'said', 'year' and 'use' rank near the top for several classes because they are frequent, not because they are discriminative. Business is the largest class with a broad vocabulary, so ambiguous articles tend to tip towards it – exactly the over-prediction seen in the confusion matrix. SVM, being discriminative, learns to ignore words shared by all classes.

Decision Tree (weakest, F1 0.8222). A tree makes a sequence of yes/no tests on single words. The fully grown tree is 35 levels deep with 135 leaves but uses only 123 of the 9,881 terms. Any article that does not contain the few words the tree happens to test (tori, game, user, film, market...) falls into a deep, narrow branch learnt from very few documents. It scores 100% on training data but only 83.1% on test data – a 17-point gap that is a textbook sign of over-fitting – and it has the largest CV variation (±0.0237). Text evidence is spread thinly over many words, which suits models that combine all features (SVM, NB, KNN) and does not suit one that picks a few hard thresholds.

6. What-If Experiments

6.1 What-if A (main): shrinking the TF-IDF vocabulary

Question: what happens if each model is only allowed to see the N most frequent terms instead of all 9,881? The vectoriser was rebuilt with max_features = 50, 100, 250, 500, 1000, 2500 and 5000, and all four models were retrained and retested each time with the same split.

max_features	Decision Tree	KNN (k=5)	SVM (Linear)	Naive Bayes
50	0.7209	0.8709	0.8792	0.8450
100	0.7850	0.9148	0.8953	0.9038

250	0.8179	0.9570	0.9618	0.9277
500	0.8170	0.9643	0.9696	0.9521
1000	0.8406	0.9713	0.9740	0.9689
2500	0.8470	0.9636	0.9719	0.9541
5000	0.8350	0.9732	0.9787	0.9537
9881	0.8222	0.9727	0.9787	0.9487

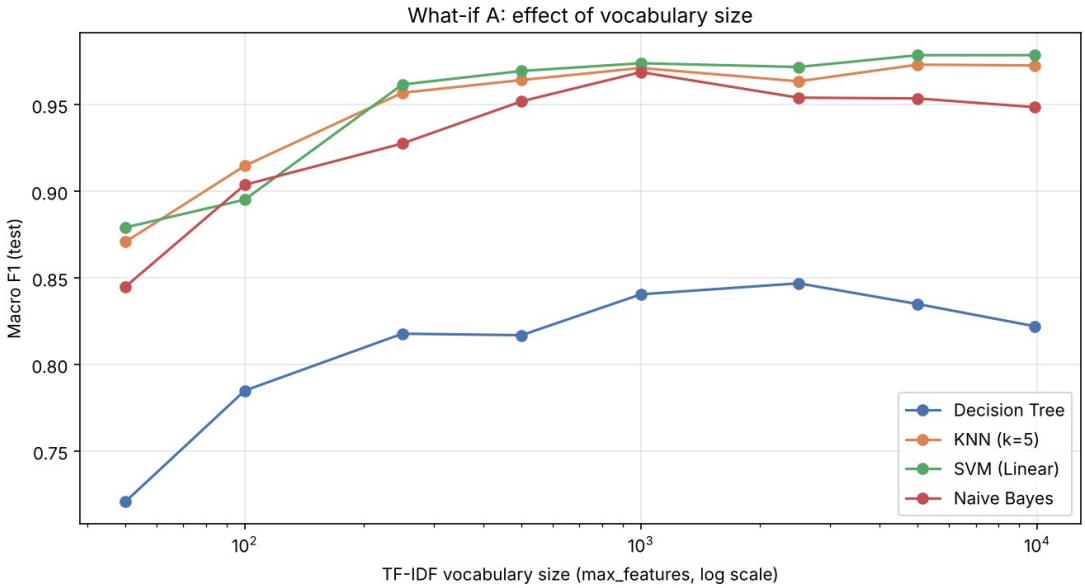


Figure 4: Macro F1 as the vocabulary size changes

- **Too few terms hurt everyone.** With only 50 terms every model loses a lot (SVM 0.879, DT 0.721). Performance climbs steeply up to about 250–1000 terms and then flattens – most of the useful signal is in the top 1,000 words.
- **Naïve Bayes improves with fewer features.** Naïve Bayes peaks at 1,000 terms (0.9689) and then drops to 0.9487 with the full vocabulary. The extra rare terms each get probability estimates from very few examples, which adds noise to its products of probabilities. Restricting the vocabulary works as feature selection for NB.
- **SVM benefits from more features.** SVM keeps improving (0.9740 → 0.9787) because its regularised weights can simply assign near-zero weight to useless terms. It is the best or joint-best model at almost every size.
- **Decision Tree stays last.** The Decision Tree is best at 2,500 terms (0.847) and is the weakest at every size, which confirms that its problem is the algorithm, not the feature set.

6.2 What-if B (supporting): KNN neighbours and distance metric

k	cosine	euclidean
1	0.9610	0.9610
3	0.9662	0.9662
5	0.9727	0.9727
7	0.9754	0.9754
9	0.9754	0.9754
15	0.9705	0.9705
25	0.9652	0.9652

51	0.9598	0.9598

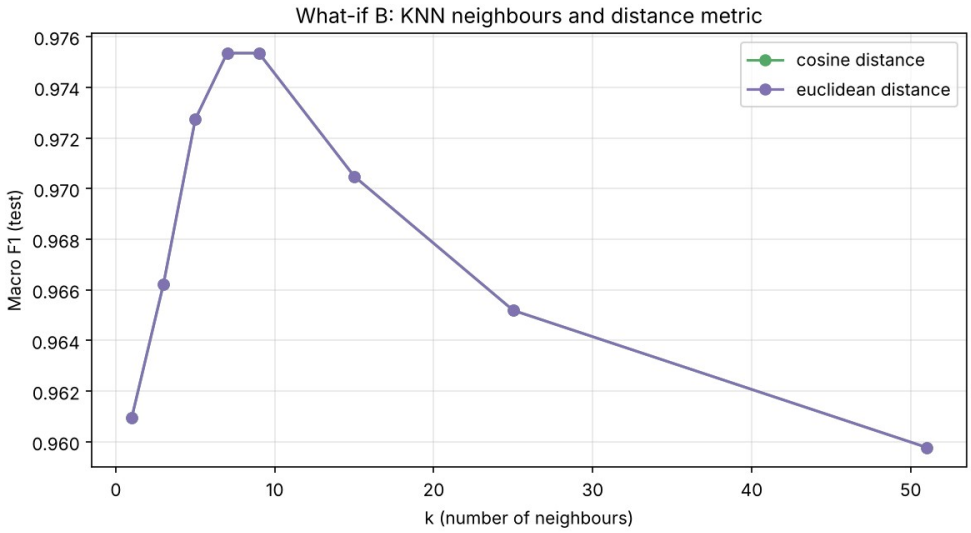


Figure 5: KNN macro F1 for different k

- **k controls the bias–variance trade-off.** k = 1 (0.9610) follows a single neighbour and is easily misled by one odd article. Accuracy rises to its best at k = 7–9 (0.9754) and falls again at k = 51 (0.9598), where the vote is so wide that the larger classes start to outvote the true one.
- **Cosine and Euclidean give the same results.** The two columns are identical. This is not a mistake: TF-IDF vectors are L2-normalised, and for unit vectors $\|a - b\|^2 = 2 - 2 \cdot \cos(a, b)$. Euclidean distance is therefore a monotonic function of cosine similarity, so both metrics choose exactly the same neighbours.

7. Final Recommendation

Linear SVM is recommended for this news-classification task. The decision rests on the measured results:

- Highest test accuracy (0.9789) and macro F1 (0.9787), with 417 of 426 articles correct.
- Highest and most consistent cross-validation score (0.9774 ± 0.0043), so the win is not an accident of one split.
- Best or joint-best F1 on four of the five classes, including a perfect 1.000 on sport.
- Prediction takes under 1 ms for 426 articles and training under 0.1 s, so it is suitable for live use.
- Its weights are interpretable – the top words per class can be shown to an editor to explain a decision.

KNN is a close second on the test split (two fewer correct articles), but it is less stable in CV and its prediction cost grows with the collection. Naïve Bayes is the best choice when training speed or very frequent retraining matters, ideally with the vocabulary limited to about 1,000 terms (F1 0.9689). A single Decision Tree is not recommended for TF-IDF text; if tree-based models are required, an ensemble such as a Random Forest would be needed to reduce its variance.

8. Limitations

- Results come from one English news corpus with five fairly distinct topics; datasets with closer classes would separate the models more.
- Only light, default-style hyper-parameters were used; tuning C, alpha or tree depth could narrow some gaps.
- Bag-of-words TF-IDF ignores word order and meaning; the errors on overlapping topics might be reduced with phrase features or embeddings.

9. Conclusion

Using one dataset, one preprocessing pipeline, one TF-IDF representation and one split, the four classifiers finished in the order SVM (0.979) > KNN (0.973) > Naïve Bayes (0.949) > Decision Tree (0.822) in macro F1. The ranking is explained by how each algorithm uses high-dimensional sparse text: models that combine evidence from many terms do well, while a tree that relies on a few hard word tests over-fits. The what-if experiments showed that Naïve Bayes gains from a smaller vocabulary while SVM gains from a larger one, and that KNN works best around k = 7–9. All numbers in this report are produced by main.py in the GitHub repository and can be regenerated.

10. References

1. D. Greene and P. Cunningham, "Practical Solutions to the Problem of Diagonal Dominance in Kernel Document Clustering," Proc. 23rd Int. Conf. on Machine Learning (ICML), pp. 377–384, 2006.
2. C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008 (Ch. 6, 13, 14, 15).
3. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
4. T. Joachims, "Text Categorization with Support Vector Machines: Learning with Many Relevant Features," Proc. ECML, 1998.
5. M. F. Porter, "An Algorithm for Suffix Stripping," Program, vol. 14, no. 3, pp. 130–137, 1980.